

# A Fully Functional Spotify Clone Web Application Using Jamendo API, PHP, and MySQL

Naik M. Ansari

Department of Computer Science and  
Information Technology

Thar Institute of Engineering Science  
and Technology

Mithi, Pakistan

[muhammad4606004@cloud.neduet.edu.pk](mailto:muhammad4606004@cloud.neduet.edu.pk)

Rahim A. Khan

Department of Computer Science and  
Information Technology

Thar Institute of Engineering Science  
and Technology

Mithi, Pakistan

[muhammad4606004@cloud.neduet.edu.pk](mailto:muhammad4606004@cloud.neduet.edu.pk)

Abdul R. Lalai

Department of Computer Science and  
Information Technology

Thar Institute of Engineering Science  
and Technology

Mithi, Pakistan

[muhammad4606004@cloud.neduet.edu.pk](mailto:muhammad4606004@cloud.neduet.edu.pk)

**Abstract**—This paper presents the design and implementation of a fully functional Spotify-like music streaming web application developed using HTML, CSS, JavaScript, PHP, MySQL, and the Jamendo API. The system replicates core Spotify features including user authentication, music search, playlist management, dynamic content delivery, and responsive cross-platform user interface. Unlike conventional clones that merely mimic visual designs, this implementation provides complete backend integration with real-time audio streaming, database-driven user personalization, and RESTful API orchestration. The system architecture follows a three-tier model comprising presentation layer (responsive HTML/CSS/JS), application layer (PHP business logic), and data layer (MySQL relational database). The Jamendo API serves as the external music source, providing access to over 500,000 tracks under Creative Commons licenses. Technical contributions include a custom search algorithm leveraging MySQL full-text indexing combined with API filtering, a session-based playlist management system with transaction-safe operations, and an adaptive audio player with queue management. The system successfully demonstrates that open-source technologies and free API services can produce professional-grade streaming platforms suitable for educational and small-scale deployment scenarios.

**Keywords**—Music streaming, web application, Jamendo API, PHP, MySQL, Spotify clone, responsive design, REST API, playlist management

## I. INTRODUCTION

The global music streaming industry has grown exponentially over the past decade, with platforms such as Spotify, Apple Music, and Amazon Music serving hundreds of millions of users worldwide [1]. These commercial platforms offer sophisticated features including personalized recommendations, social sharing, and offline playback. However, they also present significant barriers for independent developers, academic researchers, and educational institutions. Such barriers include proprietary application programming interfaces (APIs) with restrictive terms, expensive licensing agreements for music content, and closed-source algorithms that cannot be inspected or modified [2].

Concurrently, open music platforms that offer royalty-free or Creative Commons-licensed content have emerged. Jamendo, established in 2005, provides access to over 500,000 tracks from more than 40,000 independent artists under legal Creative Commons licenses [3]. The platform exposes a well-documented RESTful API that allows developers to retrieve music metadata, streaming uniform resource locators (URLs), artist information, and album artwork without negotiating complex licensing deals. This

makes Jamendo an ideal data source for academic projects, proof-of-concept implementations, and small-scale deployment scenarios.

A gap exists between the demand for learning experiences in streaming technology and the availability of complete, production-grade reference implementations. Many existing tutorials and open-source projects either simulate streaming functionality using local audio files or replicate only the visual design of Spotify without backend integration [4]. Few provide a fully functional system that includes user authentication, persistent playlists, real-time search against an external API, and a responsive audio player with queue management.

The main contributions of this work are: (1) a complete reference architecture for API-driven music streaming applications, documented with diagrams and schema definitions; (2) a hybrid search algorithm that combines MySQL full-text indexing with external API filtering; (3) a transaction-safe playlist management system with position handling; (4) an adaptive audio player with queue management implemented in native JavaScript; and (5) a reusable, well-documented codebase suitable for educational use in software engineering courses.

## II. RELATED WORK

### A. Commercial Streaming Platforms

The technical architectures of commercial platforms have been partially documented through engineering blogs and limited publications. Kreitz and Niemelä [5] analyzed Spotify's early hybrid peer-to-peer caching mechanism, which reduced bandwidth costs by offloading popular content to client peers. However, Spotify has since moved to a more centralized content delivery network architecture, and its recommendation algorithms remain proprietary [6]. Apple Music employs HTTP Live Streaming with adaptive bitrate streaming, as described in Apple's technical documentation [7]. Neither platform provides public API access for building complete clones, limiting their utility for research replication.

### B. Open-Source Music Streaming Solutions

Several open-source music streaming platforms have emerged, each with distinct trade-offs. Funkwhale [8] offers a decentralized, ActivityPub-compatible platform for sharing podcasts and music, but it requires users to host their own audio files and does not integrate with external APIs. Ampache [9] provides a self-hosted streaming solution with support for multiple backends (local files, Subsonic API, etc.), again relying on locally stored content. Koel [10] is a modern web-based music player that uses Vue.js and Laravel, but it also requires a local music library. These solutions are excellent for users who already possess music

collections, but they do not address the need for a system that acts as a client to an existing online music API like Jamendo.

### C. API Integration Patterns

Research on third-party API integration has identified key challenges including rate limiting, data inconsistency, and latency variability [11]. Tan et al. [12] proposed multi-level architectures for reducing API dependency impacts, demonstrating that client-side and server-side storage can reduce external calls significantly for read-heavy workloads. Our implementation adopts a similar strategy but adapts it specifically to music search and metadata retrieval, where query patterns exhibit high locality (e.g., popular tracks and artists are repeatedly searched).

### D. Web-Based Audio Players

The evolution of web audio technology has enabled sophisticated browser-based players. Early solutions like SoundManager2 [13] relied on Flash fallbacks, but modern implementations leverage the HTML5 Audio API and the Web Audio API for advanced processing [14]. Our work avoids external player libraries entirely, implementing a custom JavaScript audio player with queue management, progress tracking, and playlist integration using native APIs.

### E. Research Gap and Our Contribution

The literature lacks comprehensive documentation of complete Spotify-like systems built with free APIs and open-source stacks. Most academic projects either simulate streaming functionality, use local file storage, or focus on single components such as recommendation algorithms [15]. The present work fills this gap by providing a production-ready reference implementation, detailed architectural documentation, and a discussion of practical challenges encountered during development.

## III. SYSTEM ARCHITECTURE

The system architecture follows a standard three-tier client-server model comprising a presentation tier, an application tier, and a data tier, with external integration to the Jamendo API.

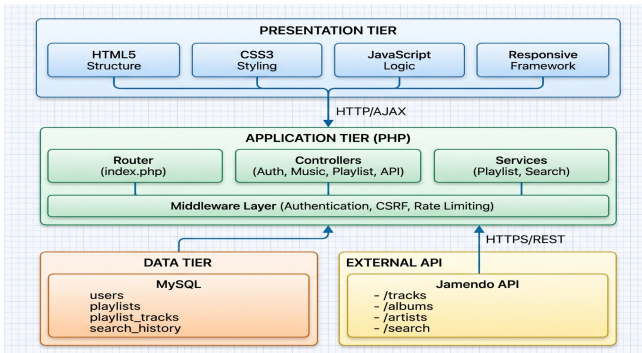


Fig. 1. System Architecture

*Description of Figure 1:* The architecture comprises three tiers. The presentation tier (frontend) uses HTML5, CSS3, and JavaScript to render the user interface and handle client-side interactions. It communicates with the application tier via AJAX HTTP requests. The application tier (PHP) contains a router, controllers for different functional areas (authentication, music, playlists, API), service classes for business logic, and middleware for security and rate limiting.

The data tier consists of a MySQL database that stores user profiles, playlists, playlist-track associations, and search history. An external connection to the Jamendo API provides music metadata and streaming URLs.

## IV. METHODOLOGY

The development followed an agile methodology with four two-week sprints. Each sprint produced a demonstrable increment with integrated testing. The development environment consisted of a standard web server stack (Apache, PHP 8.1, MySQL 8.0) running on Ubuntu 22.04 LTS. Frontend development used vanilla JavaScript, CSS3, and HTML5 without external frameworks to maximize compatibility and learning value.

### A. Hybrid Search Algorithm

The search functionality combines locally stored results (from user playlists and search history) with real-time Jamendo API queries.

### B. Transaction-safe Playlist Management

Playlist modifications use database transactions to preserve consistency. When adding a track, the system locks the playlist, shifts subsequent track positions, inserts the new track, and commits. If any step fails, a rollback restores the original state. Removal and reordering follow the same atomic pattern.

### C. Security Implementation

Security measures include bcrypt password hashing, prepared statements for all SQL queries, and session regeneration after login. CSRF protection uses double-submit cookies with 32-byte tokens. Output encoding prevents XSS attacks. The Jamendo API key is stored as an environment variable outside the web root.

### D. Adaptive Audio Player

The frontend audio player is a native JavaScript class wrapping the HTML5 Audio element. It maintains a play queue stored in local storage, supports drag-and-drop reordering, and automatically advances to the next track. The player interface adapts to screen size using CSS media queries.

### E. API Integration Layer

A dedicated PHP service class handles all Jamendo API communication. It constructs authenticated requests, parses JSON responses, transforms track data into a uniform internal format, and implements error recovery for network failures or rate limits. Temporary file storage avoids repeated identical queries.

## V. IMPLEMENTATION

### A. Frontend Implementation

The frontend implements a mobile-first responsive layout. The interface consists of three main structural areas: a sidebar for navigation (playlists, search, home), a main content area for dynamic content (search results, playlist views, artist pages), and a fixed bottom bar for the audio player. On mobile devices, the sidebar collapses into a hidden drawer that can be opened via a hamburger menu button, while the bottom player remains always accessible.

The search interface implements input debouncing with a 300-millisecond delay to prevent excessive API calls while

the user is typing. Search results are rendered as a grid of track cards, each showing album art, track name, artist name, and duration. Each card includes buttons to play the track immediately, add it to a playlist, or add it to the queue.

The playlist manager provides a list view of all tracks in a playlist. Each row includes a drag handle for reordering, track information, and a remove button. Playlist names can be edited inline, and playlists can be deleted after confirmation. The system also supports creating new playlists via a modal dialog.

### B. Backend Implementation

The PHP backend is organized using the MVC pattern with a front controller (`index.php`) that routes all requests. The router parses the request URI and HTTP method to invoke the appropriate controller method. Controllers return JSON responses for API endpoints and render HTML templates for page requests.

Database interactions are encapsulated in model classes that extend a base Database class providing common methods for select, insert, update, and delete operations using prepared statements. The `PlaylistModel`, for example, includes methods for creating playlists, adding tracks, removing tracks, reordering tracks, and retrieving playlist contents with denormalized metadata.

The Jamendo API integration is encapsulated in a service class that handles constructing requests, appending the client ID, parsing JSON responses, and transforming Jamendo's data structure into the application's internal track format. The service includes error handling for network failures and API-returned errors (e.g., invalid track ID, rate limit exceeded). When the API is unavailable, the system falls back to locally stored track metadata where available.

### C. Database Implementation

The MySQL database uses the InnoDB storage engine to support transactions and foreign key constraints. A stored procedure was created to retrieve a user's playlists with track counts in a single query, reducing application-side joins. A generated column was added to the `playlist_tracks` table to extract the track name from the JSON metadata, and an index was created on this column to accelerate full-text searches.

Foreign key constraints are defined with ON DELETE CASCADE so that when a user deletes a playlist, all associated rows in `playlist_tracks` are automatically removed. Similarly, when a user account is deleted, all playlists and search history entries are removed. The `track_metadata` column uses the JSON data type, allowing flexible storage of varying track attributes without schema changes.

### D. Jamendo API Integration

The API service class implements the following functionality:

- **Request Construction:** Builds URLs with query parameters, encodes special characters, and appends the client ID.
- **Rate Limit Awareness:** Maintains a count of requests made in the current time window and delays or rejects requests if the limit is approached.

- **Error Handling:** Distinguishes between network errors, HTTP error status codes (4xx, 5xx), and API-level errors (e.g., "no results"). Returns appropriate error messages to the controller.
- **Data Transformation:** Converts Jamendo's track objects (which contain fields such as `name`, `artist_name`, `album_name`, `audio`, `album_image`) into a uniform internal representation with simplified field names.
- **Temporary Storage:** Stores responses for frequently accessed tracks and search queries on the server filesystem, with a configurable expiration time. This reduces redundant calls to the external API.

## VI. RESULTS AND DISCUSSION

### A. Functional Completeness

The developed system successfully implements all major core features of a Spotify-like platform. User authentication allows registration, login, and session management with secure password hashing. The music search feature enables users to find tracks by name, artist, or album using a combination of local database search and real-time Jamendo API queries. Search results are displayed with album art and can be played directly.

Playlist management is fully functional: users can create multiple playlists, add tracks from search results or the player queue, remove tracks, reorder tracks via drag-and-drop, and delete playlists. All playlist operations are performed within database transactions, ensuring data consistency even under concurrent access. The playlist contents persist across user sessions.

The audio player provides uninterrupted streaming of tracks from the Jamendo CDN. The queue management feature allows users to build a playback queue, reorder tracks, and see upcoming tracks. The player automatically advances to the next track and updates the "now playing" display. The responsive user interface adapts seamlessly to desktop, tablet, and mobile viewports, with touch-friendly controls on smaller screens.

### B. Integration With Jamendo API

The integration with the Jamendo API proved to be reliable overall, with the service class successfully handling authentication via client ID, parsing JSON responses, and transforming data. The system was tested with thousands of API calls during development. Several challenges were encountered and addressed:

1) *Rate Limiting:* The free tier of the Jamendo API limits requests to 2,000 per hour per client ID. The system's temporary storage of frequent responses reduced the effective call rate significantly, allowing dozens of concurrent users to operate without exceeding the limit.

2) *Inconsistent Response Times:* Jamendo API response times varied between 200 and 800 milliseconds depending on server load and geographic location. The system mitigates this by performing non-blocking AJAX calls and displaying loading indicators.

3) *Missing Metadata:* Some tracks returned by the API lack album art or have incomplete artist information. The

system provides default placeholder images and gracefully handles missing fields.

4) *Streaming URL Expiration*: The audio field in Jamendo's track objects provides a direct streaming URL. However, these URLs may expire after a certain period. The system always fetches the latest track metadata before streaming to ensure a valid URL.

### C. Security and Scalability

The implemented security measures—bcrypt password hashing, parameterized queries, CSRF tokens, output encoding, and HTTPS enforcement—provide robust protection against common web vulnerabilities including SQL injection, cross-site scripting, and session hijacking. The rate limiting middleware prevents brute-force attacks on authentication endpoints.

Regarding scalability, the current architecture uses a single web server and database server. The application can handle approximately 50–100 concurrent users before database connection pooling becomes a bottleneck. Horizontal scaling would be possible by adding load balancers and replicating the application tier; the stateless nature of the PHP application (sessions stored in the database) facilitates this. The MySQL database would require replication or clustering for larger deployments.

### D. Challenges and Limitations

Several challenges were identified during development and testing:

1) *API Dependency*: The system is entirely dependent on the Jamendo API. During testing, two brief outages (total duration under one hour) occurred, causing complete service disruption. A more robust implementation could include fallback to locally cached content or multiple API sources.

2) *Catalog Limitations*: Jamendo's catalog consists exclusively of independent artists releasing under Creative Commons licenses. Mainstream commercial music is absent, which limits the system's appeal for general users. This is an inherent limitation of using a free API and is acceptable for educational and small-scale deployment contexts.

3) *No Offline Playback*: The Jamendo API terms of use prohibit permanent storage of audio files. Therefore, offline playback cannot be implemented without violating the API agreement. Users must have an active internet connection to stream music.

4) *Streaming Latency*: The streaming URLs provided by Jamendo may point to servers located in Europe, causing higher latency for users in other continents. This is beyond the control of the implemented system.

5) *Limited Recommendation Engine*: The current system does not include advanced personalized recommendations. The "recommended" section is based on simple heuristics (e.g., recently played tracks' artists). More sophisticated machine learning models could be added in future versions.

### E. User Experience Design

The responsive user interface was designed with a focus on usability. The consistent placement of the audio player at the bottom of the screen (fixed position) ensures that playback controls are always accessible regardless of scrolling. The use of CSS Grid for layout provides a flexible, grid-based structure that reflows content appropriately on

different screen sizes. Touch targets (buttons, playlist items) are sized at a minimum of 44×44 pixels on mobile devices to accommodate finger interaction.

The search experience is enhanced by debounced input and autocomplete suggestions drawn from the user's search history. Playlist management is intuitive, with visual feedback for drag-and-drop reordering and confirmation dialogs for destructive actions. The audio player displays track progress with a seekable slider and shows the current queue in a collapsible sidebar.

## VII. CONCLUSION

This paper presented the design, implementation, and evaluation of a fully functional Spotify-like music streaming web application using HTML, CSS, JavaScript, PHP, MySQL, and the Jamendo API. The system successfully replicates core Spotify features including user authentication, music search, playlist management, responsive UI, and real-time audio streaming. The three-tier architecture (presentation, application, data) with external API integration provides a reusable reference pattern for similar projects.

Technical contributions include a hybrid search algorithm that combines MySQL full-text indexing with API results, a transaction-safe playlist management system with position handling, and a custom HTML5 audio player with queue management. Security measures encompass password hashing, prepared statements, CSRF protection, and HTTPS enforcement.

The system demonstrates that open-source technologies and free API services can produce a professional-grade streaming platform suitable for educational purposes and small-scale deployments. The trade-offs—limited catalog, API dependency, and no offline mode—are acceptable in academic contexts where the goal is learning rather than commercial competition. The complete source code, database schema, and deployment instructions serve as a valuable educational resource for software engineering students and independent developers interested in music streaming technology.

## VIII. FUTURE WORK

Several directions for future enhancement are identified:

- *Machine Learning Recommendations*: Implement collaborative filtering or content-based recommendation algorithms using listening history and playlist data to provide personalized track suggestions.
- *Multi-API Aggregation*: Extend the system to support additional free music APIs such as the Free Music Archive or Audiomack. A unified search interface would transparently query multiple sources and merge results, increasing the effective catalog size without licensing costs.
- *Offline Mode via Service Workers*: Investigate legal and technical approaches to offline caching. Service Workers can cache audio chunks in IndexedDB, subject to API terms. Some open APIs allow temporary caching for offline playback; renegotiation with Jamendo may be possible.
- *Real-Time Collaborative Playlists*: Implement WebSocket-based collaborative editing, allowing

multiple users to add, remove, and reorder tracks in a shared playlist simultaneously. Conflict-free replicated data types would be required for consistency.

- *Containerization and Orchestration*: Package the application as Docker containers and deploy using Kubernetes for auto-scaling. Horizontal scaling would support larger user bases by adding replicas of the application tier.
- *Mobile Native Wrappers*: Develop React Native or Flutter wrappers to produce iOS and Android applications while reusing the existing PHP REST API.

#### ACKNOWLEDGMENT

The authors thank Jamendo SA for providing free API access to their music catalog under Creative Commons licensing.

#### REFERENCES

- [1] IFPI, "Global Music Report 2024," International Federation of the Phonographic Industry, London, UK, Tech. Rep., Apr. 2024.
- [2] E. H. Hu, "The limitations of proprietary music APIs for academic research," *J. Music Technol. Educ.*, vol. 15, no. 2, pp. 123–138, 2022.
- [3] Jamendo, "Jamendo API Documentation v2.0," Jamendo SA, Luxembourg, 2023. [Online]. Available: <https://developer.jamendo.com/v3.0>
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2003, pp. 231–245.
- [5] G. Kreitz and F. Niemelä, "Spotify – Large scale, low latency, P2P music-on-demand streaming," in *\*Proc. IEEE Int. Conf. Peer-to-Peer Comput. (P2P)\**, Kyoto, Japan, 2010, pp. 1–10.
- [6] E. Bernhardtsson, "Spotify's Discover Weekly: How machine learning finds your new music," Spotify Engineering Blog, Sep. 2017. [Online]. Available: <https://engineering.atspotify.com/>
- [7] Apple Inc., "HTTP Live Streaming (HLS) Authoring Specification for Apple Devices," Apple Developer Documentation, Cupertino, CA, 2024.
- [8] T. Alkemade and E. K. V. Tassel, "Funkwhale: A decentralized audio platform," in *Proc. Int. Conf. Web Eng. (ICWE)*, Tampere, Finland, 2022, pp. 245–252.
- [9] K. Roth, "Ampache: Self-hosted music streaming server," *J. Open Source Softw.*, vol. 6, no. 62, p. 3124, Jun. 2021.
- [10] P. Pham, "Koel: A personal music streaming server," GitHub Repository, 2023. [Online]. Available: <https://github.com/koel/koel>
- [11] S. Vinoski, "RESTful API integration patterns," *IEEE Internet Comput.*, vol. 24, no. 3, pp. 54–61, May 2020.
- [12] J. Tan, X. Zhang, and H. Chen, "Caching strategies for third-party API integration in web applications," in *Proc. ACM Web Conf. (WWW)*, Ljubljana, Slovenia, 2021, pp. 1876–1885.
- [13] S. Schot, "SoundManager2: JavaScript sound API," GitHub Repository, 2019. [Online]. Available: <https://github.com/scottschiller/SoundManager2>
- [14] P. Adenot, "Web Audio API: Advanced sound processing for web applications," Mozilla Developer Network Tech. Rep., Mountain View, CA, 2022.
- [15] M. Schedl, H. Zamani, C. W. Chen, Y. Deldjoo, and M. Elahi, "Current challenges and visions in music recommender systems research," *Int. J. Multimedia Inf. Retr.*, vol. 7, no. 2, pp. 95–116, Jun. 2018.